

/bash

Manage bash command lists: `/bash add|remove allow|deny [--global] | show | help`

Overview

/bash maintains the user-defined allowlist and denylist consumed by the bash tool's layered command security model. Allowlist entries are command prefixes that bypass the built-in banned-command check, so a single command such as `curl` can be re-enabled without turning on YOLO mode. Denylist entries are substring patterns that always reject a command, no matter what other rules say. Every change is persisted immediately and takes effect at once: project scope writes `.ragent/ragent.json`, global scope writes `~/.config/ragent/ragent.json`.

Syntax

```
/bash add allow <entry> [--global]
/bash add deny <entry> [--global]
/bash remove allow <entry> [--global]
/bash remove deny <entry> [--global]
/bash show
/bash help
```

Notes:

- `--help` and `-h` also print the help page.
- The `--global` flag may trail the entry; it is trimmed off the entry text and redirects the write from the project config (`.ragent/ragent.json`) to the user config (`~/.config/ragent/ragent.json`).
- An empty `<entry>` prints the usage line instead of storing an empty record.

Options / Subcommands

Form	Description
<code>/bash add allow <entry> [--global]</code>	Add a command prefix to the allowlist. Commands starting with the entry are no longer blocked by the built-in banned-command check.
<code>/bash add deny <entry> [--global]</code>	Add a substring pattern to the denylist. Commands containing the pattern are always rejected.
<code>/bash remove allow <entry> [--global]</code>	Remove an entry from the allowlist.
<code>/bash remove deny <entry> [--global]</code>	Remove an entry from the denylist.
<code>/bash show</code>	Print the full security list report (see Output).
<code>/bash help</code>	Print the help page.

Examples

Re-enable `curl` in the project scope (no more banned-command block):

```
/bash add allow curl
```

Block any command containing `rm -rf`:

```
/bash add deny rm -rf
```

Remove an allowlist entry from the user-global config instead of the project config:

```
/bash remove allow curl --global
```

Inspect every list the bash security model consults:

```
/bash show
```

Empty entry prints the usage line and stores nothing:

```
/bash add allow
```

Unknown list type prints the correction hint:

```
/bash add foo curl
```

Output

`/bash show` prints the seven sections below, in this order. Empty user lists print `*(empty)*`.

1. Built-in Safe Commands - Layer 1, auto-approved without a prompt.
2. Allowlist - user-defined, Layer 2 exemptions. Your bypass prefixes.
3. Denylist - user-defined, Layer 3 custom blocks. Your substring patterns.
4. Built-in Banned Commands - Layer 2, word-boundary matched. Blocked unless the command is allowlisted or YOLO mode is on.
5. Built-in Denied Commands - Layer 3, unconditional rejects (for example `mkfs`, `insmod`, `useradd`).
6. Built-in Denied Command Patterns - Layer 3, command-with-arguments patterns (for example `sudo`, `su -`, `passwd`).
7. Built-in Denied Patterns - Layer 3, substring matched.

Message formats:

- Successful add: `[ok] Added ... Commands starting with X will no longer be blocked by the banned-command check`
- Remove of an entry that is not present: `[warn] ... was not in the {scope} allowlist/denylist`
- Backend failure: `[err] Error: {e}`
- Empty entry: the usage line.
- Unknown list type (anything other than allow/deny): Use `allow` or `deny`
- Unknown subcommand: Run `/bash help`

Related

- Directory and file path lists: `/dirs` (same add/remove/show model).
- Permission rules and YOLO mode: `docs/howtos/permissions.md`.
- The bash tool's 7-layer security model: safe-command whitelist, banned commands, denied patterns, and the user allow/deny layers above.
- Config files written: `.ragent/ragent.json` (project), `~/.config/ragent/ragent.json` (global).